C++ Mastery Course In Hindi

Topics :— https://www.youtube.com/@webdevmastery18

1. Introduction to C++
2. Structure of a C++ Program
3. Data Types & variables
4. Input/output (cin, cout)
5. Operators → [Arithmetic, Relational, Logical, Bitwise]
6. Conditional Statements → [if, if-else, switch-case]
7. Loops → [for, while, do-while]
8. Function & Function Overloading
9. Introduction to Arrays
10. strings & string functions
11. Pointers & Pointer Arithmetic
12. Dynamic Memory Allocation → [new, delete]
13. Structure & Unions
14. Recursion
15. File Handling
16. Preprocessor Directives → (#define, #include)
17. Enumerations → (enum, enum class)
18. Classes & Objects
19. Constructors & Destructors
20. Access Specifiers → (public, private, protected)
21. this Pointer
22. static Members
23. Friend functions & classes
24. Inheritance → (Single, Multiple, Multilevel, Hierarchical, Hybrid)
25. Function Overriding
26. Polymorphism → (Compile-time & Run-time)
27. Virtual functions & Pure Virtual functions (Abstract class)
28. Operator Overloading
29. Exception Handling → (try, catch, throw)
30. STL → Standard Template Library → [Vectors, set, Maps---]

Scanned with CamScanner

Date: / / Page:

1. Introduction to C++ :-

What is C++ ?

C++ is a high-level, object-oriented programming language developed by "Bjarne Stroustrup" at Bell Labs as an enhancement to the C language.

C++ is used in :-

(I) Game development
(II) System/software development
(III) Competitive programming
(IV) Performance-critical applications

Features of C++ :-

(I) Fast & efficient
(II) Supports OOP (classes, Objects, Inheritance, Polymorphism)
(III) Portable & Platform Independent
(IV) Rich Standard Template Library (STL)
(V) Memory Management with Pointers

Practice Questions :-

(I) Write a program to display :- "Hello World"

(II) Write a program to display your name on the screen.

Scanned with CamScanner

Page :

(2.) Structure of a C++ Program :-

(A) Components of a C++ program

```
#include<iostream>   // Preprocessor Directive
using namespace std;  // Standard namespace

int main(){           // Entry point of the program
    cout<<"Hello World";  // output statement
return 0;             // End of the program
    }
```

(A) Detailed Explanation :-

(I) `#include<iostream>` :- Includes the input/output stream from the standard library.

(II) `using namespace std;` - Allows usage of standard names like cout, cin without std:: prefix.

(III) `int main()` - Main function from where execution starts.

(IV) {...} - Curly braces define the body or scope of the function

(V) `cout<<` - Used to output/print on the console.

(VI) `return 0;` - Return 0 to indicate successful execution.

(A) Practice Questions :-

(I) Write a program to print "I am learning C++".

(II) Identify & explain each part of the following C++ program structure.

Scanned with CamScanner

③ Data Types & Variables :-

Data Type in C++

Basic Data Types
- int
- float
- double
- char
- bool
- void

Derived Data Types
- array
- pointer
- reference
- function

User Defined Data Types
- class
- structure
- union
- typedef
- using

④ Variables :-

Variables are containers used to store the data in memory. Each variable has data type, which defines the type of value it can hold.

⑤ Built-in Data Types :-

Data Type	Example	Description
int	10	Integer (no decimal)
float	3.14	Decimal (single precision)
double	3.141592	Decimal (double precision)
char	'A'	character
bool	true/false	Boolean value

Scanned with CamScanner

Note:-

(I) int (integer) stores whole numbers [-∞, +∞]
(II) Float & double stores decimal numbers.
(III) char (character) stores single character
(IV) bool (Boolean) stores true/false.

(*) Example:-

```
#include <iostream>
using namespace std;

int main() {
    int age = 22;
    float weight = 55.5;
    char grade = 'A';
    bool isStudent = true;

cout << "Age:" << age << endl;
cout << "Weight:" << weight << "kg" << endl;
cout << "Grade:" << grade << endl;
cout << "Is Student:" << isStudent << endl;

return 0;
}
```

(*) Practice Questions:-

(I) Write a program to declare a variable for your age, name, CGPA.

(II) Write a program to display your name, age, & whether you are a student or not using variables.

Scanned with CamScanner

Date / / Page

④ Input/Output (cin, cout) :-

(A) Output using cout :-

(i) cout is used to display output on screen.
(ii) It uses the insertion operator <<.

```
cout << "Hello World";
```

(B) Input using cin :-

(i) cin is used to take input from user.
(ii) It uses the extraction operator >>.

```
int age;
cin >> age;
```

(A) Example :-

```
#include <iostream>
using namespace std;
int main(){
    string name; int age;

    cout << "Enter your name: " << endl;
    cin >> name;

    cout << "Enter your age: " << endl;
    cin >> age;

    cout << "Hello " << name << ", you are " << age << "
        years old." << endl;

    return 0;
}
```

Scanned with CamScanner

Note:-

cin with >> can only take single-word strings.
To take full sentences, use getline().

Example:-

```
int main(){

string fullName;

cout<<"Enter your full name:";
getline(cin, fullName);

cout<<"You entered: "<<fullName<<endl;

return 0; }
```

Practice Questions:-

(i) Write a program that takes your full name & prints it.

(ii) Write a program that takes 2 numbers & displays their sum.

(iii) Write a program that asks for age & prints:

You are __ years old.

Scanned with CamScanner

Date: / / Page

(5.) Operators :-

Operators are special symbols used to perform operations on variables & values. Below are the categorized of operators in C++ :-

(a) Arithmetic Operators :-

Used to perform mathematical operations.

	operator	Description	Example
(i)	+	Addition	a+b
(ii)	-	Subtraction	a-b
(iii)	*	Multiplication	a*b
(iv)	/	Division	a/b
(v)	%	Modulus(remainder)	a%b

(b) Assignment Operators :-

Used to assign values to variables.

	Operator	Description	Example
(i)	=	Assign	a=b
(ii)	+=	Add & assign	a+=b
(iii)	-=	Subtract & assign	a-=b
(iv)	*=	Multiply & assign	a*=b
(v)	/=	Divide & assign	a/=b
(vi)	%=	Modulus & assign	a%=b

Scanned with CamScanner

(c) Relational (Comparison) operators:-

Used to compare two values.

	Operator	Description	Example
(i)	==	Equal to	a==b
(ii)	!=	Not equal to	a!=b
(iii)	>	Greater than	a>b
(iv)	<	Less than	a<b
(v)	>=	Greater than or equal	a>=b
(vi)	<=	Less than or equal	a<=b

(d) Logical Operators:-

Used to combine multiple conditions.

	Operator	Description	Example
(i)	&&	Logical AND	a>0 && b>0
(ii)	!	Logical NOT	!a
(iii)	||	Logical OR	a>0 || b>0

(e) Increment/Decrement Operators:-

Used to increase or decrease a variable's value.

	operator	Description	Example
(i)	++	: Increment (by 1)	++a, a++
(ii)	--	Decrement (by 1)	--a, a--

Scanned with CamScanner

Date: / / Page

(f) Bitwise Operators :-

Operate on bits & perform bit-by-bit operations.

	Operator	Description	Example
(i)	^	XOR	a^b
(ii)	~	NOT	~a
(iii)	&	AND	a&b
(iv)	|	OR	a|b
(v)	<<	Left Shift	a<<1
(vi)	>>	Right Shift	a>>1

(g) Conditional (Ternary) Operator :-

Syntax :- condition ? expression 1 : expression 2

Example :- int a = (x>y) ? x : y // max of x & y

(h) Sizeof Operator :-

Returns the size of a data type.

Example :-

```
Sizeof(4);
Sizeof("Ram");
```

(i) Scope Resolution Operator (::) :-

Used to access global variables or class members.

Example :- ::x // accesses global x.

Scanned with CamScanner

Date: / / Page

Practice Questions :-

(i) Write a program to perform all arithmetic operations on two user-input numbers.

(ii) Write a program to check if a number is even or odd using % operator.

(iii) Write a program to check if two numbers are equal or not.

(iv) Write a program to check if both numbers are positive using logical operator.

(v) Write a program to demonstrate the use of bit-wise AND and OR operators.

Scanned with CamScanner

⑥ Conditional Statements :-

Conditional statements allow the program to make decisions based on certain conditions.

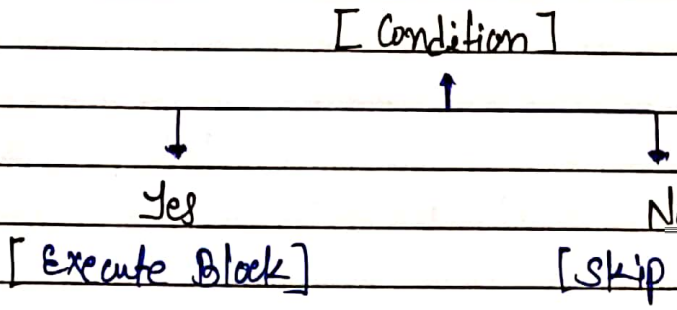
(i) if - statement :-

Executes a block of code if the condition is true.

code :-

```
int age = 20;

if (age >= 18) {
    cout << " Eligible to vote";
}
```

(*) Flowchart :-



(ii) if - else statement :- When you want to execute one of two blocks based on a condition.

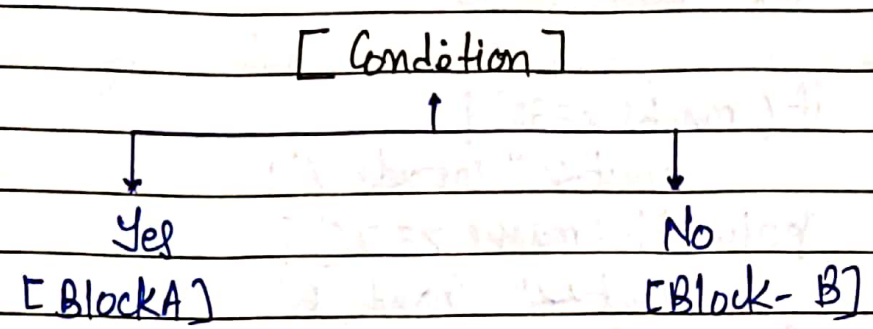
Syntax :-

```
if (condition) {
// code if true
} else {
// code if false
}
```

Scanned with CamScanner

Date: / / Page.

Flowchart :-



Example :-

```
int marks = 45;

if( marks >= 50){

   cout<< "Pass";
}
   else{
   cout<< "fail";
}
```

(iii) else- if Ladder :-

Used to check multiple conditions.

Syntax :-

```
if ( condition 1) {
   // block 1
} else if (condition 2) {
   // block 2
} else {
     // block 3
  }
```

Scanned with CamScanner

Code:-

```
int marks = 85;

if ( marks >=90){
    cout<< " Grade A";
}else if (marks >=75){
    cout<<" Grade B";
}else if( marks >=60){
    cout<<" Grade C";
}else
    cout<<"Fail";
```

(iv) Nested if statement:-

Syntax:-

```
if (condition 1){

if(condition 2){
  // nested block
}
}
```

Example:-

```
int a=10, b=5;

if (a>0){

if( b>0){
    cout<<"Both are positive";
}
}
```

Scanned with CamScanner

⑦ switch statement :-

Used when you want to compare a single value against multiple constants.

⑧ Syntax :-

```
switch (expression){

case value1:
    //code
    break;
case value2:
    //code
    break;
default:
    // default code

}
```

⑨ Example :-

```
int day = 2;

switch (day){

case 1: cout<<"Monday"; break;
case 2: cout<<"Tuesday"; break;
case 3: cout<<"Wednesday"; break;
case 4: cout<<"Thursday"; break;
case 5: cout<<"Friday"; break;
case 6: cout<<"Saturday"; break;
case 7: cout<<"Sunday"; break;
default: cout<<"Invalid day";
```

Scanned with CamScanner

Date: / / Page

Practice Questions :-

(i) Write a program to check whether a person is eligible to vote or not (age ≥ 18).

(ii) Write a program to check whether a number is even or odd using ternary operator.

(iii) Write a program to input marks & print the grade using if-else conditions.

(iv) Write a program to find the largest among the three numbers.

(v) Write a program to build a simple calculator. It should perform (+, -, *, /).

(vi) Write a program to input a character & check whether it is a vowel or consonant.

Scanned with CamScanner

Date: / / Page:

⑦ Loops:-

Loops help reduce redundancy by running a block of code multiple times

⭐ Why use Loops:-

without loops:-

```
cout << "Hello";
cout << "Hello";
cout << "Hello";
```

With loops:-

```
for (int i = 0; i < 3; i++){
    cout << "Hello";
}
```

⭐ Types of Loops:-

Loop Type	Entry/Exit Controlled	When to Use
① for loop	Entry-Controlled	When number of iterations is known
⑪ while loop	Entry-Controlled	When iterations depend on a condition
⑪ do-while	Exit-Controlled	When loop must runs at least once

Scanned with CamScanner

Date: / / Page

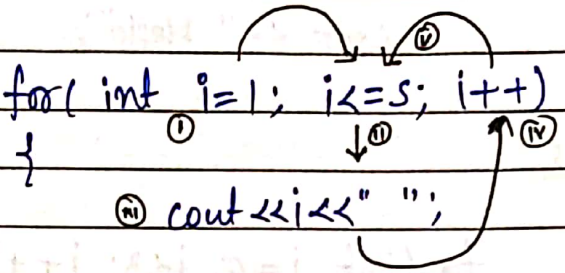
(a) **for loop :-**

When the number of iterations is known

(*) **Syntax :-**

```
for (initialization; condition; updation){
    // loop body
}
```

(*) **Example :-**



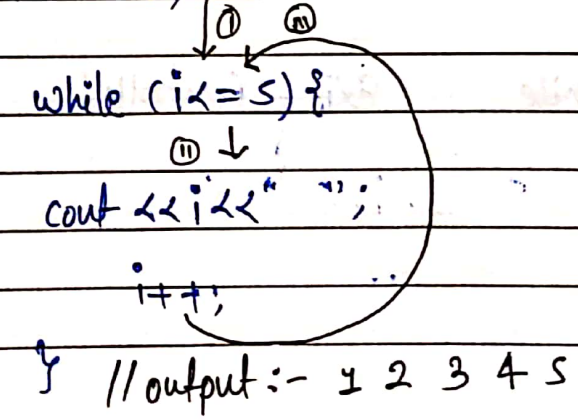
(b) **while loop :-**

When the number of iterations is Unknown.

(*) **Syntax :-**

```
while (condition){
    // loop body
}
```

(*) **Example :-**



Scanned with CamScanner

ⓒ do-while loop :-

When you want the loop to run at least once, without checking the condition.

✱ Syntax :-

```
do {
    // loop body
} while (condition);
```

✱ Example :-

```
int i = 1;
do {
    cout << i << " ";
    i++
} while (i <= 5);

// output :- 1 2 3 4 5
```

✱ Loop Control Statements :-

① break statements :-

ⓐ Terminates the loop immediately

ⓑ Common in switch or to exit easy from loops.

✱ Example :-

```
for(int i=1; i<=5; i++)
{
    if(i==3) break;
    cout<<i<<" ";
}
// output: 1 2
```

Scanned with CamScanner

(ii) continue Statement :-

Skips the current iterations & jumps to the next iterations.

code :-

```
for (int i=1; i<=5; i++){
    if (i==3) continue;
    cout<<i<<" ";
}
//output: 1 2 4 5
```

(iii) Practice Questions :-

(i) Write a program to print numbers from 1 to N.

(ii) Write a program to calculate the sum of first N natural numbers.

(iii) Write a program to find factorial of given number.

(iv) Write a program to print the first N terms of the fibonacci series.

(v) Write a program to check given number is prime or not.

(vi) Write a program to count the number of digits in an integer.

Scanned with CamScanner

(VII) Write a program to find the Sum of the digit of a number.

(VIII) Write a program to input a number & print its multiplication table

Ex:- 5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
:
5 x 10 = 50

(IX) Write a program to reverse the digits of a number.

ex:- 123 → 321

(X) Print the following pattern.

1
1 2
1 2 3
1 2 3 4

Scanned with CamScanner

Nested Loops :-

A nested loop is a loop inside another loop.

Syntax :-

```
for(int i=1; i<= outer_limit; i++){

    for(int j=1; j<= inner_limit; j++){

        // code to execute
    }
}
```

Note :- The inner loop runs completely for every single iteration of outer loop.

Flow Control Diagram :-

Outer Loop [i=1]
↳ Inner Loop [j= 1 to N]
↳ Outer Loop [i=2]
↳ Inner Loop [j=1 to N]
...

Example :-

```
for(int i= 1; i<=3; i++){

    for(int j=1; j<=2; j++){

        cout<<"("<<i<<","<<j<<")";
    }
    cout<<endl;
}
```

Scanned with CamScanner

Date: / / Page

④ Output :-

(1,1) (1,2)
(2,1) (2,2)
(3,1) (3,2)

① Square Pattern of Stars :-

```
* * * *
* * * *
* * * *
* * * *
```

```
int n=4;
for(int i=1; i<=n; i++){
for(int j=1; j<=n; j++){
  cout<<"*";
}
  cout<<endl;}
```

② Right-Angled Triangle Pattern :-

```
*
* *
* * *
* * * *
* * * * *
```

```
int n=5;
for(int i=1; i<=n; i++)
{ for(int j=1; j<=i; j++)
  {
    cout<<"*";
} cout<<endl;
}
```

③ Number Triangle :-

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

```
int n=5;
for(int i=1; i<=n; i++)
{
  for(int j=1; j<=i; j++)
{ cout<<j<<" ";
}
  cout<<endl;
}
```

Scanned with CamScanner

Date: / / Page

⭐ Practice Questions :-

(i) Hollow Rectangle :-

```
* * * * *
*       *
*       *
* * * * *
```

(ii) Right - Angled Triangle of Numbers :-

```
1
2 2
3 3 3
4 4 4 4
```

(iii) Inverted Triangle of Stars :-

```
* * * * *
* * * *
* * *
* *
*
```

(iv) Multiplication Table Grid (1 to 5) :-

```
1  2  3  4  5
2  4  6  8  10
3  6  9  12 15
4  8  12 16 20
5  10 15 20 25
```

Scanned with CamScanner

⑧ Functions :-

Functions help you break down a program into manageable, reusable blocks of code. They improve modularity, reusability, & readability.

⑨ What is Function :-

A function is a block of code that performs a specific task. It runs only when it is called.

⑩ Types of functions :-

Type	Description
① Built-in Functions	Predefined (e.g. count, cin, sqrt())
② User-defined Functions	Created by the programmer

⑪ Structure of a User-Defined Function :-

Syntax :-

```
// function declaration (prototype)
returnType functionName (parameter list);

// function definition
returnType functionName (parameter list) {
    // function body
}

// Function call
functionName (arguments);
```

Scanned with CamScanner

Date: / / Page

Example: Sum of Two Numbers :-

```
#include<iostream>
using namespace std;

int add(int a, int b){  // Function definition
    return a+b;
}

int main(){
    int result = add(5,3);  // Function call
    cout<<"Sum:"<<result;
}
```

Function Components :-

(i) Return type :- Type of value returned (int, float, void etc)

(ii) Function name :- Any valid identifier

(iii) Parameter :- input (optional)

(iv) Function body :- Code to execute

(v) Return statement :- Sends back result (except in void)

Function Declaration (Prototype) :-

Used before main() to inform the compiler about the function.

```
int add(int, int); // declaration (no body)
```

Scanned with CamScanner

⭐ Types of User-Defined functions:-

Type	Syntax example
(i) No argument, no return type	void greet();
(ii) Argument, no return type	void display(int);
(iii) No argument, return type	int getInput();
(iv) Argument, return	int square(int);

⭐ Call by Value vs Call by Reference:-

(i) Call by Value (default):-

Passes a copy. Original value not changed

```
void fun(int x){ x = x+10; }
```

(ii) Call by Reference:-

Passes the address. Original value gets modified.

```
void fun(int &x){ x = x+10; }
```

⭐ Default Parameters:-

You can provide default values:-

```
int add(int a, int b=10) {
    return a+b;
}
```

Scanned with CamScanner

⭐ Function Overloading :-

C++ allows multiple functions with the same name but different parameters.

Code :-

```
int area(int side) {return side * side;}

int area(int l, int b) {return l * b;}
```

⭐ Practice Questions :-

(I) Write a functions to add two numbers
(II) Write a functions to find factorial of given number.
(III) Write a function to check whether a number is prime.
(IV) Write a function to reverse a number.
(V) Write a function to swap two numbers using call by reference.
(VI) Write a function to check if a string is a palindrome.
(VII) Write a function that takes two arguments & returns the maximum.
(VIII) Overload a function area() to calculate :-
  (i) Area of square.
  (ii) Area of Rectangle
  (iii) Area of circle

Scanned with CamScanner

Date: / / Page

⑨ Introduction to Arrays:—

An array is a collection of elements of the same data type, stored at contiguous memory locations

✱ What is an Array:—

(i) A fixed-size sequential collection of elements of the same type.

(ii) Index starts from 0.

(iii) Memory-efficient & fast for lookup.

✱ Array Declaration & Initialization:—

Syntax:—

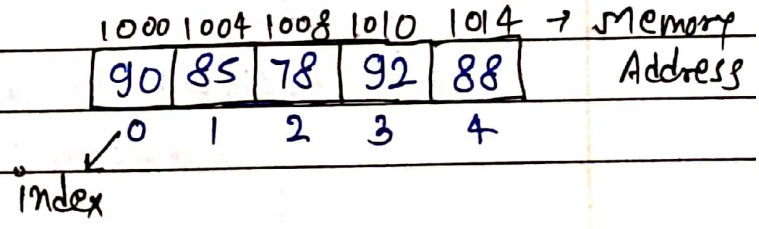
```
datatype arrayName [size];
```

Example:—

```
int arr[5];          // Declaration only
int marks[5] = {90, 85, 78, 92, 88}; // Declaration + Initialization
int num[] = {1,2,3}; // Size inferred as 3
```

→ `int marks[5] = {90,85,78,92,88};`

✱ Memory representation:—



Scanned with CamScanner

Accessing elements :-

```
int marks[5] = {90,85,78,92,88};
```

marks[0] → 1st element
marks[n-1] → last element

code :- `cout << marks[2]; // output 78`

Size of array :-

```
int marks[] = {10,20,30,40};
int length = sizeof(marks) / sizeof(marks[0]);
```

16 (size of int) / 4 (size of first element)

= 16/4

= 4. Ans.

Input & output in Array :-

code :-

```
int arr[5];
for(int i=0; i<5; i++){
    cin>>arr[i];
}
for(int i=0; i<5; i++){
    cout<<arr[i]<<" ";
}
```

Scanned with CamScanner

⭐ Types of Arrays :-

① One-Dimensional Array :-

```
int arr[10]; // Single row of elements
```

② Two-Dimensional Array (Matrix):-

```
int matrix[3][4] // 3 rows, 4 colums
```

③ Multidimensional Array (3D, 4D..):-

```
int space[2][3][4] // 3D Array
```

⭐ Example :- 2D-Arrays:-

```
int matrix[2][3] = {{1,2,3}, {4,5,6}};

for(int i=0; i<2; i++){
    for(int j=0; j<3; j++){
        cout<<matrix[i][j]<<" ";
    }
    cout<<endl;
}
```

⭐ Passing Arrays to Functions :-

⭐ 1D Array :-

```
void printArray(int arr[], int size){

    for(int i=0; i<size; i++){
        cout<<arr[i]<<" ";
    }
}
```

Scanned with CamScanner

2D Array :-

```
void print_matrix(int mat[][3], int rows){
    for(int i=0; i<rows; i++){
        for(int j=0; j<3; j++){
            cout<< mat[i][j]<<" ";
        }
    }
}
```

Why mat[][3] when passing to functions :-

(i) The number of columns must be fixed because C++ needs to calculate memory layout.

(ii) It must know how many columns to jump to go from one row to the next.

Explanation :-

mat[i][j] is accessed as:-

*(mat + i * columns + j)

So, the number of columns must be fixed in the function declaration.

Scanned with CamScanner

Practice Questions :-

(i) Write a program store 10 integers & print them in reverse order.

(ii) Write a program to find the second largest number in a array.

(iii) Count the number of even & odd integers in array.

(iv) Write a program to find the sum of all elements in a array.

(v) Write a program to check if an array is palindrome or not

(vi) Find the maximum & minimum element in array.

(vii) For a 2D array, print the sum of each row & each column.

(viii) Merge the two array into 1.

(ix) Search for an element in array (return index if found).

(x) Find Avg. of array elements.

Scanned with CamScanner

(10) Strings :- string is a sequence of characters.

(1) What is std::string:-

(a) std:: is a class in the c++ Standard Library.
(b) It is part of the Standard Template Library (STL).

(*) Header:-

#include<string>

(*) Internally:-

(i) It manages its own memory.
(ii) Stores characters internally in a dynamically allocated memory (array)
(iii) A null character "\0" is stored internally, but you don't manage it manually - unlike char[]

(*) c++ vs c String Memory:-

Aspect	char[] (c-style)	std::string (c++style)
(i) NULL terminator	Must add '\0' Manually	Added automatically, (hidden)
(ii) Manual size tracking	Yes	No (size managed internally)
(iii) Resizing	Not possible	Dynamic resizing
(iv) Memory safe?	x Risky (buffer overrun)	safer
(v) operator overloads	x	+, ==, etc. supported

Scanned with CamScanner

⍟ Declaration & Initialization :-

```
#include<string>
using namespace std;

int main(){

string s1;                  // empty string
string s2 = "Hello";        // initialized with literal
string s3 = "Ram";          // constructor style → ⍟
string s4("World");         // constructor style
```

⍟ Input & Output :-

① Input :-

```
cin>>s;                     // Read until space
getline(cin,str);           // Read entire line
```

⑪ Output :-

```
cout<<str;
```

⍟ Internal Handling of \0 :-

Yes, std::string internally adds a null-terminator \0, but :

① It is used for compatibility with C-style functions (like c_str()).

⑪ It is not counted in s.length().

⑪ You don't see or manage it manually.

Scanned with CamScanner

Date: / / Page

Example :-

```
string s = "soman";

cout << s.length(); // 5
cout << s.c_str(); // converts to char* (with '\0')
```

String Functions :-

```
s.size();      // same as length()
s.length();    // number of visible characters
s.capacity(); // allocated memory
s.empty();     // true if string is empty
```

Modifiers :-

```
s.clear();              // erase content
s.append("World");      // add to end
s.insert(2, "++");      // insert at index
s.replace(2,3,"AB");    // replace 3 chars at index 2
s.erase(0,2);           // erase 2 chars from index 0
```

Searching :-

```
s.find("Man");    // returns index
s.rfind("Man");   // last occurrence
```

Substrings :-

```
s.substr(2,4);  // start at index 2, length 4
```

Scanned with CamScanner

⛤ Character Access :-

```
string s= "Suman";

cout << s[0];        // S
cout << s.at(1);     // u
cout << s.front();   // S
cout << s.back();    // n
```

⛤ Reversing a String :-

```
# include <algorithm>

reverse ( s.begin(), s.end());
```

⛤ Iteration :-

```
for (char ch: str) cout << ch << " ";
```

OR :-

```
for( int i=0; i< s.size(); i++) cout<< s[i];
```

⛤ Conversion Between int & string :-

```
string s= to_string (123);  // "123"
int x = stoi ("456");       // 456
```

Scanned with CamScanner

✡ Comparisons using Overloaded Operators (==, <, >) :-

Code:-

```
string a = "abc", b = "abd";

if(a == b) cout << "Equal";
if(a < b) cout << "a comes before b";
```

✡ How it works :-

In C++, the string class from the STL (std::string) overloads comparison operators like ==, <, > etc., to work intuitively with strings.

(i) ==: Checks if strings are exactly equal character by character.
(ii) <: Compare lexicographically (like a dictionary - character by character from left to right).
(iii) >: Also lexicographical comparison, but checks if one string comes after another.

✡ Example :-

```
string a = "abc";
string b = "abd";

if(a == b)
    cout<<"Equal"; // Will not print because "abc" ≠ "abd"

if(a < b)
    cout << "a comes before b"; // will print because
                                 "abc" < "abd"
```

Scanned with CamScanner

Date: / / Page:

Why "abc" < "abd" :-

(i) Compare first character. 'a' == 'a'
(ii) Compare second character. 'b' == 'b'
(iii) Compare third character. 'c' < 'd' → "abc" < "abd"

This is called lexicographical comparison - like how words are arranged in a dictionary.

Using .compare() Method :-

Syntax :-

```
a.compare(b); // returns :
              // 0  → if (a == b)
              // <0 → if (a < b)
              // >0 → if (a > b)
```

How it works :-

The .compare() function is a member function of std::string. It also compares lexicographically.

(i) If strings are equal → returns 0
(ii) If 'a' comes before 'b' → returns negative value
(iii) If 'a' comes after 'b' → returns positive value

code :-
```
string a = "abc", b = "abd";

int result = a.compare(b);
if(result == 0) cout << "equal";
else if(result < 0) cout << "a comes before b";
     else cout << "a comes after b";
```

output :- a comes before b;

Scanned with CamScanner

Date / / Page

⭐ Interview-Level Practice Questions :-

(i) Reverse a string using only std::string functions.

(ii) Count vowels & consonants in a string.

(iii) Check if a string is a palindrome.

(iv) Remove all digits from a string.

(v) Remove duplicate characters from a string.

(vi) Replace spaces with hyphens.

(vii) Find the first non-repeating character.

(viii) Count frequency of each character in a string.

Scanned with CamScanner

(11) Pointers & Pointer Arithmetic :-

(I) What is a Pointer :-

A pointer is a variable that stores the memory address of another variable.

In c++, every variable is stored at some memory address. A pointer allows indirect access to the memory.

(*) Why Use Pointers :-

(i) Efficient memory access (low-level control)
(ii) Used in dynamic memory allocation (new, delete)
(iii) Needed for arrays, strings, & functions (pass by ~~value~~)
(iv) Base of linked lists, trees, & graphs

(*) Declaring & Using Pointers :-

Syntax :-

```
int *ptr; // pointer to int
```

Example :-

```
int x=10;
int *ptr = &x;

cout << "Address of x:" << &x << endl; // memory address
cout << "Value at ptr:" << *ptr << endl; // 10
```

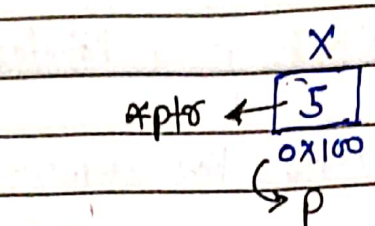
(i) &x :- address-of operator
(ii) *ptr :- deference pointer (get value stored at that address)

Scanned with CamScanner

Date: / / Page:

④ Memory Layout Illustration :-

```
int x=5;
int *p = &x;

// Suppose x is at memory 0x100
// Then p= 0x100, *p =5
```



Visual :-

x: 5 (at address 0x100)
p: 0x100 (points to x)
*p: 5 (value at address p)

⑤ Types of Pointers in C++ :-

	Type	Description
(i)	int*	Pointer to int
(ii)	char*	Pointer to char
(iii)	float*	Pointer to float
(iv)	void*	Generic pointer (can't be directly dereferenced)
(v)	nullptr	A pointer with no address
(vi)	int**	pointer to pointer

⑥ Pointer Arithmetic :-

Pointer arithmetic used the fact that each data type occupies memory.

Scanned with CamScanner

⭐ Pointer Arithmetic:-

Pointer arithmetic uses the fact that each data type occupies memory.

code:-

```
int arr[3] = {10,20,30};
int *p = arr;

cout << *p << endl;    //10
cout << *(p+1) << endl; //20
```

(p+1) moves ahead by sizeof(int) (typically 4 bytes)

⭐ Pointers with Arrays:-

In c++, the name of an array is a pointer to its first element.

code:-

```
int arr[4] = {1,2,3,4};
int *p = arr;

for(int i=0; i<4; i++){
    cout << *(p+i) << " ";
}
```

Scanned with CamScanner

Date: / / Page:

✡ Pointers & Functions :-

① Pass-by-value :-

```
void change(int x) {
    x = 100; // only local copy changes
}
```

⑪ Pass-by-pointer (original changes) :-

```
void change(int *p) {
    *p = 100;
}
int main() {
    int a=5;
    change(&a);
    cout<<a;   //100
}
```

✡ nullptr – Safer NULL Pointers :-

```
int *p = nullptr; // safer than NULL
if(p == nullptr) {
    cout<<"Pointer is null";
}
```

Prevents error from uninitialized or dangling pointer

Scanned with CamScanner

Date: / / Page

④ void* - The Generic Pointer :-

① What is void* :-

ⓐ A generic pointer that can hold address of any data types.
ⓑ Cannot be directly dereferenced (you must cast it).

⑪ Why Use void* :-

ⓐ For generic functions, like memory allocators (malloc)
ⓑ For type-independent data handling.

④ Example :-

```
void *ptr
int x = 42;
ptr = &x;

cout << *(int*)ptr; // must typecast to correct type
```

Note :- Without typecasting, dereferencing a void* is a compile-time error.

⑤ Double Pointer (int**) :-

A pointer that holds the address of another pointer.

⑤ Use-case :-

① Dynamic 2D arrays

⑪ Modifying a pointer inside a function.

Scanned with CamScanner

Date: / / Page

Example :-

```
int x=5;
int *p = &x;
int **pp = &p;

cout<<**pp //5
```

Practice Questions :-

(i) Swap two numbers using pointers.

(ii) Implement a function that changes value via pointer.

(iii) Create dynamic array, input values, print sum.

(iv) Implement a function that returns max of two numbers using pointers.

(v) Use pointer arithmetic to reverse an array.

Scanned with CamScanner

⁂ Dynamic Memory Allocation :-

c++ provides two major ways to allocate memory dynamically at runtime :-

⁂ C - style (from C) :-

(i) malloc() :- memory allocation
(ii) calloc() :- contiguous allocation & zero - initialization
(iii) realloc() :- resize previously allocated memory.
(iv) free() :- deallocation

These come from <stdlib> header & are still valid in c++, but not preferred.

⁂ c++ style :-

(i) new :- allocates memory & calls constructor

(ii) delete :- deallocates memory & calls destructor.

⁂ new & delete in c++ :-

⁂ Why Use :-

(i) Allocate memory when size isn't known at compile time.

(ii) Create objects that need to live beyond their scope.

(iii) Manage resources manually (e.g. in system programming, embedded, game dev.)

Scanned with CamScanner

④ How to Use:-

① single variable:-

```
int *ptr = new int // Allocates space for 1 int

*ptr = 10;
cout << *ptr; // Output: 10

delete ptr; // Frees the memory
```

⑪ Array allocation:-

```
int * arr = new int [5]; // Allocates array of 5 ints

for( int i=0; i<5; i++){
    arr[i] = i+1;
}

delete [] arr; // Deallocates array
```

⑤ How delete Works:-

```
delete ptr; // deallocates single value
delete [] arr; // deallocates array
delete obj ; // deallocates objects
```

Scanned with CamScanner

Page \ \ Date: / / Page.

④ Complete Example:-

```
# include <iostream>
using namespace std;

int main() {

// Allocate a single integer dynamically
   int *num = new int;
      *num = 100;
cout << "value: " << *num << endl; // output: 100

// Allocate an array of integers dynamically

   int *arr = new int[3];
     arr[0] = 10;
     arr[1] = 20;
     arr[2] = 30;

cout << "Array:";
   for (int i=0; i<3; i++) {
      cout << arr[i] << " ";
   }
     cout << endl;

// Deallocate memory

   delete num;      // free single int
delete[] arr;       // free array
   return 0;
   }
```

Scanned with CamScanner

Date: / / Page:

(13) Structure & Union :-

① **Structure :-** In C++, a struct (Short of structure) is a user-defined data type that allows you to group different type of variables under a single name.

Note :- It's commonly used to represent real-world entities like a student, book, employee, product etc.

② **Syntax :-**

```
struct StructName {
    dataType member1;
    dataType member2;
    ...
};
```

③ **Why use struct :-**

	Purpose	Benefit
(i)	Group related data	keep logically related data together
(ii)	Organized Code	Improve readability & structure
(iii)	Easy Access	Access members with a single variable
(iv)	Lightweight	No need for object-oriented features when not required.

Scanned with CamScanner

Example :-

```
#include <iostream>
using namespace std;

struct Student {          // Structure definition
    int roll;
    string name;          } => Structure Member
    float marks;

    void display_student() { // function to display student info

        cout << "Roll Number = " << roll << endl;
        cout << " Name      = " << name << endl;
        cout << " marks     = " << marks << endl;
}
};

int main() {

Student s1 = ;  // Declare structure variable

s1. roll = 101;      // Assign values
s1. name = "Suman";
s1. marks = 92.05;

s1. display_student(); // Display the structure values.

return 0;
}
```

Scanned with CamScanner

Date: / / Page

⭐ Struct vs class :-

Features	struct (structure)	class
(i) Default access	Public	Private
(ii) Use case	Light-weight data	Full OOP design
(iii) Member functions	Allowed	Allowed

(ii) Union :-

A union is a user-defind data type in C++ (similar to struct) that allows you to store different types of data in same memory location.

⭐ Note :-

All members of a union share the same memory space, so only one member can hold a value at a time.

Syntax :-

```
union UnionName{

    dataType member1;
    dataType member2;
    ...
};
```

⭐ Why use union :-

Purpose	Benefit
(i) Save Memory	Only the largest member size is allocated
(ii) Used for multi-purpose data	When a variable needs to store one value from several types.

Scanned with CamScanner

Date: / / Page:

✡ When to use Union:-

① You need to store only one value at a time from multiple variables.
② You want to save memory, especially in memory-constrained environments.
③ You're dealing with low-level data representation like interpreting bits or bytes.

✡ Example :-

```
# include <iostream>
using namespace std;

union Data{
  int intValue;
float floatValue;
char charValue;
};
  int main(){

  Data d;
  d.intValue = 10; cout<<"intValue:" <<d.intValue<<endl;
d.floatValue = 5.5; cout <<"floatValue:" <<d.floatValue <<endl;
d.charValue = 'A'; cout<<"charValue:"<<d.charValue <<endl;

// check values again
cout<<" After assigning charValue:"<<endl;
 cout<<" intvalue:"<< d.intValue <<endl; // may be garbage
cout << "floatValue:"<< "d.floatValue <<endl; // may be garbage
 cout<<" charValue:"<<d.charValue<<endl;

  return 0;
}
```

Scanned with CamScanner

Date: / / Page:

⭐ Union vs Structure :-

Features	struct	union
(i) Memory Allocation	Separate memory for each member	Shared memory (common space)
(ii) Store multiple values?	Yes, all members can have values	No, only one member at a time
(iii) Size of type	Sum of all member sizes	Size of the largest member
(iv) Use case	When all fields are needed simultaneously	When only one field is used at a time
(v) Example use	Student data, Employee record	Variant data, Embedded systems

⭐ Memory Example :-

```
struct mystruct {
    int a;    // 4 bytes
    float b;  // 4 bytes
    char c;   // 1 byte
};
```

⇒ Total : ⇒ 9 bytes

```
union MyUnion {
    int a;    // 4 bytes
    float b;  // 4 bytes
    char c;   // 1 byte
};
```

Total: 4 bytes [largest member only]

Scanned with CamScanner

Date: / / Page:

(14) Recursion :-

Recursion is a programming technique where a function call itself, either directly or indirectly, to solve a problem.

Note:- It works by breaking a complex problem into smaller subproblems of the same kind & solving each using the same logic.

✱ Real-Life Analogy :-

(i) Imagine placing a set of mirrors facing each other - each mirror reflects the reflection endlessly.

(ii) In recursion, a function keeps calling itself until a base condition is met.

✱ Structure of a Recursive Function :-

```
returnType functionName (parameters) {

    if (base_condition) {
        return base_result; // base case to stop recursion
    }
    else {
        // smaller subproblem
        return functionName (smaller_problem);
    }
}
```

Note :- (i) Base Case :- Stops recursion (MUST have this).
(ii) Recursive Case :- Function calls itself with smaller input.

Scanned with CamScanner

Date: / / Page:

✡ Types of Recursion in C++ :-

① Tail Recursion
② Head Recursion
③ Tree Recursion
④ Direct Recursion
⑤ Indirect Recursion

① Tail Recursion :-

There is recursive call is the last statement in the function.

✡ Example:-

print 1 to n.

Code :-

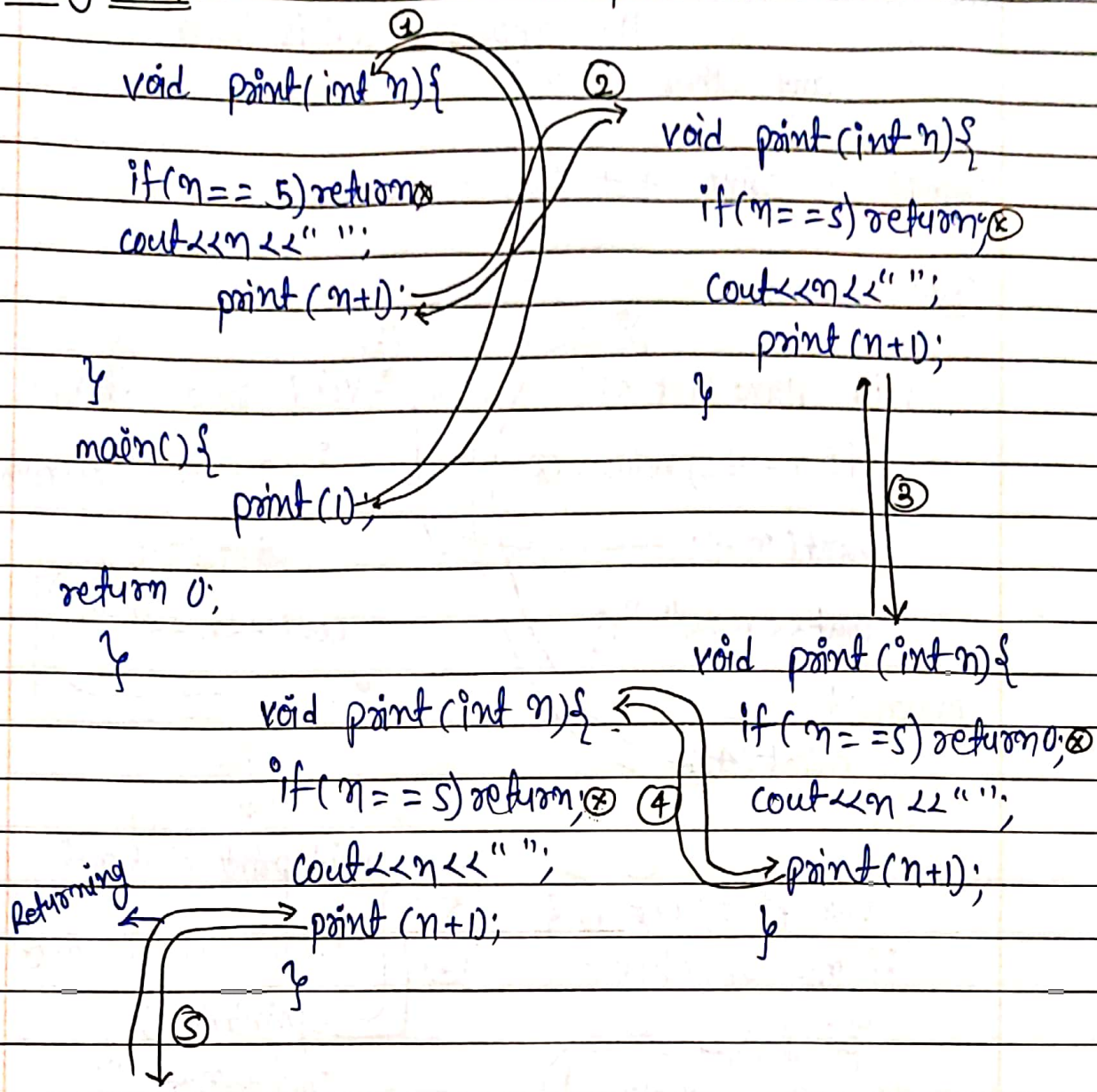
```
#include<iostream>
using namespace std;

Void tailRec (int n){

    if(n==5) return;
    cout<<n<<" ";

    tailRec(n+1);
}
int main() {

    int n=1;

    tailRec (n);

return 0;
}
```

Scanned with CamScanner

Date: / / Page:

Dry Run:-

output:- 1 2 3 4



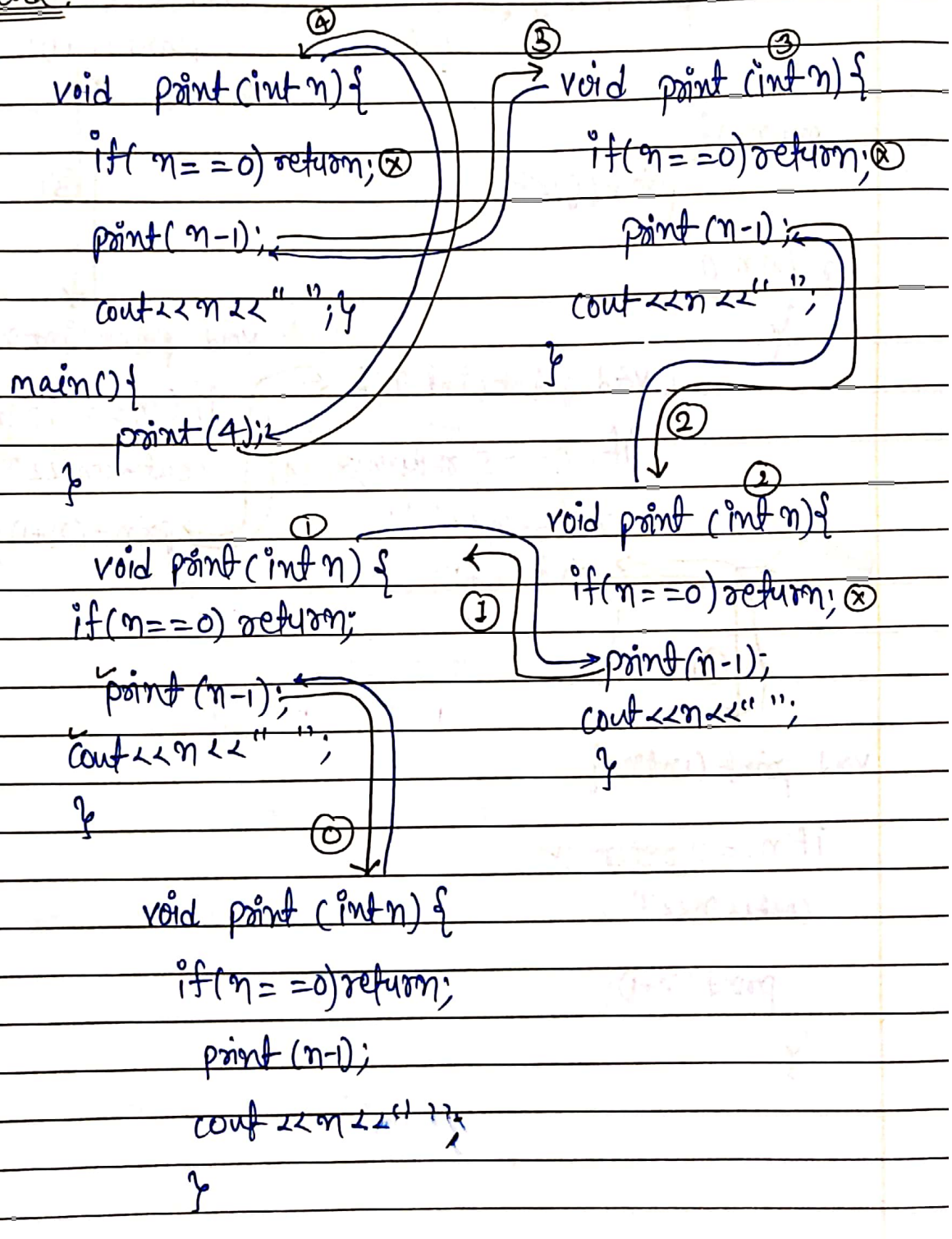
Scanned with CamScanner

(11) Head Recursion :-

The recursive call is before any other operation.

Example :- print 4 to 1.

Code :-



Scanned with CamScanner

(iii) Tree Recursion :-

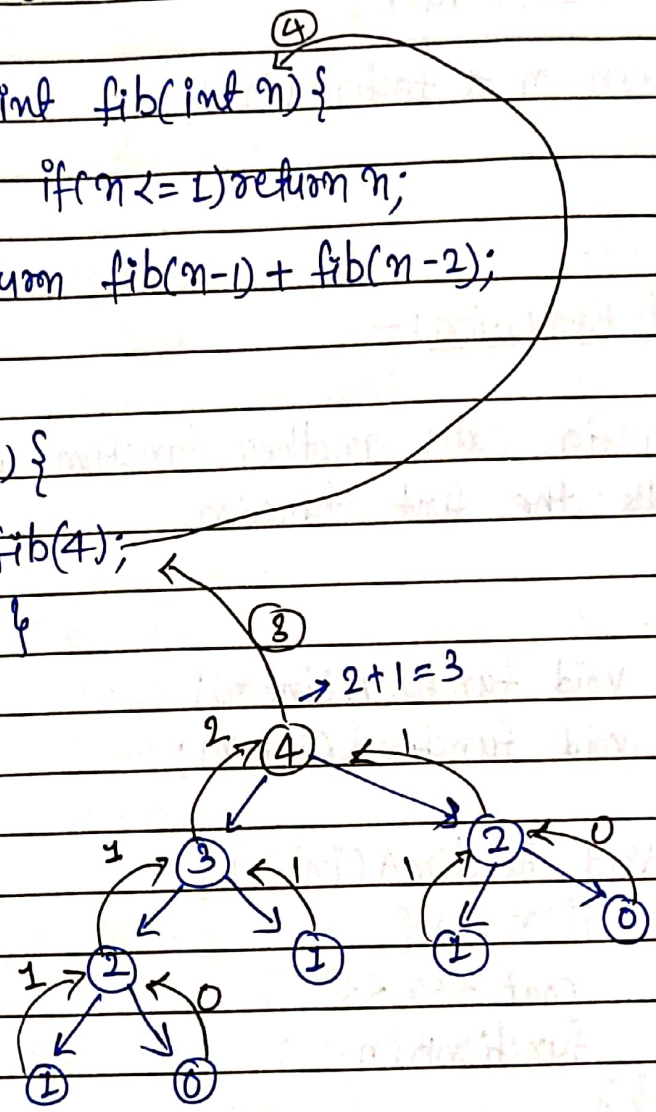
A function make more than one recursive call.

Example :- Fibonacci Series

0 1 1 2 3 5 8 13 21

(iv) Code :-

```
int fib(int n){
    if(n<=1) return n;
return fib(n-1) + fib(n-2);
}

int main(){
  cout<<fib(4);
return 0; }
```



Scanned with CamScanner

(iv) Direct Recursion:-

A function calls itself directly.

Example:- Factorial

```
int factorial (int n){
    if(n==0) return 1;
    return n * factorial(n-1);
}
```

(v) Indirect Recursion:-

A function calls another function, which in turn calls the first function.

Example:-

```
void functionA(int n);
void functionB(int n);

void functionA(int n){
    if(n>0){
    cout<<n<<" ";
    functionB(n-1);
}}

void functionB(int n){
    if(n<0){
    cout<<n<<" ";
    function(n/2);
}}
```

Scanned with CamScanner

Date: / / Page:

✡ How Recursion Works Internally:-

Each function call is placed on the call stack. Once the base case is reached, the stack start to unwind, & the results are combined (if needed).

✡ Recursion vs Iteration (loop):-

Feature	Recursion	Iteration
(i) Simplicity	Cleaner for complex logic	Usually longer but efficient
(ii) Speed	slower (stack overhead)	Faster (no stack usage)
(iii) Memory	Uses more memory (stack)	constant memory,
(iv) Risk	stack overflow on large input	No overflow risk
(v) Suitable for	Trees, backtracking, recursive models	Loops, arrays, counting problems

✡ Advantages of Recursion:-

(i) Clean & elegant code.
(ii) Easy to write for tree & graph problems.
(iii) Powerful for divide-&-conqur strategies.
(iv) Simplifies code for problems with subproblems.

Scanned with CamScanner

⁂ Disadvantages of Recursion:-

(i) Consumes more stack memory.
(ii) Risk of stack overflow
(iii) Can be inefficient (like in Fibonacci)
(iv) Slower than iteration for simple problems

⁂ Practice Questions:-

(1) Write a recursive function to print number from N to 1.
(2) Write a recursive function to print number from 1 to N.
(3) Write a recursive function to print factorial of given number
(4) Calculate the sum of first N natural numbers.
(5) Check if a given string is a palindrome.
(6) Print the fibonacci series.
(7) Calculate Power (x^n).
(8) Write a function to reverse a given array.
(9) Write a recursive function to calculate the sum of digits of a given number.
(10) Write a recursive function to find the Greatest Common Divisor (GCD) of two integers using the Euclidean algorithm.

Scanned with CamScanner

(15.) File Handling :-

File handling in C++ means storing data permanently in files on disk so it can be retrieved, updated, or read later, even after the program ends.

Note :- Just like variable store data temporarily in RAM, files store data permanently on disk.

⊛ Required Header File :-

→ `#include<fstream>`

This above header file provide 3 main classes for file operations :-

	Class	Purpose
(i)	ifstream	Input file stream → for reading
(ii)	ofstream	Output file stream → for writing
(iii)	fstream	File stream → for both reading & writing

⊛ Common File Operations Functions :-

	Operation	Function/class used
(i)	Create/Write	ofstream, fstream
(ii)	Read	ifstream, fstream.
(iii)	Append	ofstream, fstream (with ios::app)
(iv)	Check if Open	.is_open()
(v)	Close File	.close()
(vi)	Read Line	getline (file, str)
(vii)	Write Line	file<<"text";

Scanned with CamScanner

What is ios:: in C++:-

→ ios:: stands for Input Output Stream & it defines file modes that tell C++ how to open or operate on a file.

Common File Modes (Flags):-

	Mode	Meaning
(i)	ios:: in	Open file for reading
(ii)	ios:: out	Open file for writing
(iii)	ios:: app	Append to the end of file
(iv)	ios:: ate	Go to end of file immediately after opening.
(v)	ios:: trunc	Delete the existing content of the file
(vi)	ios:: binary	Open file in binary mode

Note:-

Q. Is ios:: Required Every Time?

Ans. No, not always.

Example:-

```
fstream fout("myfile.txt");
```

By default, it opens the file in ios:: out mode.

is the same as: [Similarly] → `ifstream fin("a.txt");`

```
ifstream fin("a.txt", ios::in);
```

Both code Return same result.

Scanned with CamScanner

But if you want multiple modes, like appending + writing, then you must use ios::

Example :-

```
ofstream fout("myfile.txt", ios::app | ios::out);
```

④ How to Create File in C++ :-

Creating a file in C++ is very simple. When you open a file using "ofstream", if the file does not exist, it will automatically create it.

Code :-

```
#include<iostream>
#include<fstream>
using namespace std;

int main(){

    ofstream file("data.txt"); // creates file if not present
    if(file.is_open()){

        file<< "File created successfully.\n";
        file.close();
        cout<<" File written.\n";
    }
    else {
        cout<<"Failed to create file.\n";
    }
    return 0;

}
```

Note :- This creates a file named data.txt & writen text into it.

Scanned with CamScanner

Date / / Page :

⊛ Writing to a File :-

```
#include<iostream>
#include<fstream>
using namespace std;

int main(){

    ofstream fout("example.txt");

    if(fout.is_open()){

        fout<<"Hello, woorld!\n";
        fout<<"Writing to a file in C++ \n";
        fout.close();
    } else
      { cout<<" Error opening file.\n"; }

    return 0;
}
```

Scanned with CamScanner

(A) Reading from a File:-

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main(){
    ifstream fin("example.txt");
    string line;

  if(fin.is_open()){
   while (getline(fin, line)){

      cout <<line <<endl;
  }
    fin.close();
}
  else{
      cout<<" File not found \n";
    }
  return 0;
}
```

Scanned with CamScanner

Date: / / Page:

⊛ Reading & Writing :-

```
#include <iostream>
#include <fstream>
using namespace std;

int main(){
    fstream file("example.txt", ios::in | ios::out);

    if(file.is_open()){

    file<<"Writing & reading in same stream.\n";

        file.seekg(0); // go to start to end to read

        string line;
            while (getline(file, line)){

                cout<<line<<endl;
            }
            file.close();
        }
        return 0;
    }
```

⊛ Practice Questions :-

(i) Write a program to create a file & store your name & roll number.

(ii) Write a program that reads a file line-by-line & prints all lines in uppercase.

Scanned with CamScanner

Date: / / Page:

(16) Preprocessor Directives in C++ :-

Preprocessor directives are commands that are executed before the actual compilation of the code begins. These commands are processed by the preprocessor, which prepares the source code for the compiler.

- All preprocessor directives begin with a # symbol,

Example :- #include, #define, #ifdef.

Why Use Preprocessor Directives:-

(i) To include files (#include).
(ii) To define constants or macros (#define).
(iii) To make code more readable & manageable.

#include - Include Header Files :-

```
#include <iostream>   // standard header
#include "myheader.h" // user-defined header
```

#define - Define Constants/Macros:-

```
#define PI 3.14159
#define SQUARE(x) ((x)*(x))
int main(){
std::cout << "Value of PI:" << PI << endl;
std::cout << "Square of 4:" << SQUARE(4) << endl;
}
```

Note:- Macros do not occupy memory like const variables.

Scanned with CamScanner

Date: / / Page:

(17) Enumerations :-

enum (enumeration) is a user-defined data type in C++ used to assign names to a set of integral constant values, making a program more readable & maintainable.

✱ Use enum when:-

(i) You have a fixed set of related constant values
(ii) You went to improve code readability.
(iii) You want to avoid magic numbers (unnamed literal constants)

✱ Syntax :-

user defint data type

```
enum EnumName { value1, value2, value3 ---};
```

✱ Example :-

```
#include<iostream>
using namespace std;

enum Day {Monday, Tuesday, Wednesday, Thursday};
int main(){
    Day today = Wednesday;

    if(today == Wednesday){

        cout<<"It's midweek\n";
    }
return 0;
}
```

Scanned with CamScanner

Internal Behavior:-

By default:-

Monday = 0
Tuesday = 1
Wedensday = 2
...

→ You can also manually assign values:-

```
enum Day { Monday = 1, Tuesday = 2, Friday = 3 };
```

enum class:-

enum class is A modern, type-safe, and scoped version of traditional enum,

Syntax:-

```
enum class Status { Pending, Success, Failed };
```

Example :-

```
#include<iostream>
using namespace std;

enum class Status { Pending, Success, Failed };

int main(){
  Status s = Status:: Success;

  if(s == Status:: Success){
    cout<<" Task was successful";
  }
return 0;
}
```

Scanned with CamScanner

Date: / / Page:

⊛ OOPs → (Object-Oriented Programming):—

Object-Oriented Programming (OOP) is a programming paradigm that organizes code using objects. It focuses on data & methods rather than procedures & functions.

Note:- C++ is one of the first languages to implement OOP as a core part of the language.

⊛ Why OOP :—

(i) Organizes large codebases
(ii) Makes code modular, reusable, & scalable
(iii) Supports real-world modeling
(iv) Helps in implementing security, abstraction & extensibility

⊛ Basic Building Blocks :—

① Class :—

A class is like a blueprint or template.

→ Think of a class as a design of a car - it defines what features the car has (engine, wheels) & what it can do (start, stop).

→ "But this design doesn't create a real car yet - it's just a plan."

Scanned with CamScanner

(ii) Object :-

An object is a real-world version of the class.

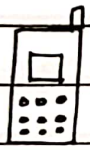
"If a class is a car design, then an object is a real car built from that design.

→ Object is an instance of a class.

(iii) 4 Main Pillars of OOP :-

(1.) Abstraction :-

Hiding the complex implementation details & showing only the necessary features or functionality.



⇒ Use this smartphone without thinking about the how it was made.

(2.) Encapsulation :-

Wrapping data & functions into a single unit (class) & restricting direct access to some of the objects.

↳ The act of putting various components together (in a capsule)

⇒ Laptop is a single entity with screen + camera + keyboard in single box [like a capsule]

Scanned with CamScanner

Date: / / Page:

③ Inheritance :-

Acquiring properties from the parent class.

" Mechanism where one class inherits properties & behaviors (methods) from another class.

Phone ⇒ Smart Phone.

④ Polymorphism :-

One entity can play multiple Role based on the context or situation.

(Greek word) ← Poly + morphism
↓ ↓
multiple form

Smartphone → Phone (calling)
Smartphone → Calculator.

⑤ Writing a Custom Class :-

```
class Employee{
    int id;  → Attribute 1
    string name;  → Attribute 2
}
```

Scanned with CamScanner

① Encapsulation :-

(i) classes & Objects
(ii) Access Specifiers (public, private, protected)
(iii) Constructors & Destructors
(iv) this pointer
(v) static Members
(vi) Friend functions.

② Abstraction :-

(i) Abstract classes
(ii) Pure virtual functions
(iii) Virtual Function
(iv) Friend functions
(v) Operator Overloading

③ Inheritance :-

(i) Inheritance [Single, Multiple, Multilevel - - -]
(ii) Constructors & Destructors
(iii) Access Specifiers

④ Polymorphism :-

(i) Function Overriding
(ii) Polymorphism (Compile-time & Run-time)

→ (a) Compile-time :- Function overloading, Operator overloading.

→ (b) Run-time :- Virtual Functions.

(iii) Virtual Functions & Pure Virtual Functions
(iv) Operator Overloading

Scanned with CamScanner

Date: / / Page:

⭐ Classes & objects :-

```
#include <iostream>
using namespace std;

class Car{
public:
    string brand;
    int speed;

    void drive(){
        cout<<" Driving "<< brand<<"at "<<speed <<"km/h";
    }
};
int main(){
    Car car1; //car1 is an object (instance) of class Car
    car1.brand = " Toyota";
    car1.speed = 100;
    car1.drive();
                    //output : Driving Toyota at 100 km/h
return 0;

}
```

Scanned with CamScanner

Date: / / Page:

(A) Access Specifiers :—

Access Specifiers control which part of the program can access the class members (variables & functions).

(B) Types of Access Specifiers :—

Specifier	Accessible Within class	outside class	In Derived class
(I) public	✓	✓	✓
(II) private	✓	x	x
(III) protected	✓	x	✓

Note :- By default, class members are private in C++.

Ex :- Class A{
int x; // this is private by default
};

(I) Public :- Open to All
Used when you want members to be accessible from anywhere.

→
```
class Student {
public:
    string name;
  void sayHello(){
    cout<<"Hi, I am"<<name<<endl;
  };
int main(){ Student S;
        S.name = "Suman"; // Accessible directly
        S.sayHello(); // Accessible directly
return 0;}
```

Scanned with CamScanner

(2.) Private :- Only inside the class

→
```
class BankAccount {
private :
        int balance;
public :
        void setBalance (int b) {
            balance = b;
        }
        int getBalance () {
            return balance;
        }
};

int main() {
```

→
```
    BankAccount b;
    // b.balance = 1000; → Error: private member

    b.setBalance (1000);

    cout<< "Balance :" << b.getBalance();

    return 0;
}
```

Scanned with CamScanner

③ Protected :- Like private, but available to child classes.

→ Used in inheritance. Accessible in the class & in derived classes, but not outside.

→
```
class Parent {
protected:
    int age = 50;
};

class Child : public Parent {
public:
    void showAge() {
        cout << "Age is " << age << endl;
    }
};
```

→
```
int main() {

    Child c;
    c.showAge();

    // cout << c.age → Error: protected member

    return 0;
}
```

Final Notes :-

① Use private for data protection (encapsulation)
② Use public to expose necessary behaviour.
③ Use protected mainly when working with inheritance.

Scanned with CamScanner

Date: / / Page:

⭐ Constructors & Destructors:-

① What is Constructor :-

A constructor is a special function that is automatically called when an object is created from a class. Its main purpose is to initialize data members of the class.

⭐ key points :-

(i) Constructor name is same as the class name.
(ii) No return type, not even void.
(iii) Automatically called when object is created.
(iv) Can be overloaded

Note :- If you don't write a constructor, C++ provide a default constructor that does nothing.

⭐ Types of Constructor :-

(i) Default Constructor
(ii) Parameterized Constructor
(iii) Copy Constructor

(i) Default Constructor:-

(i) No parameter
(ii) Automatically provided by compiler if no constructor is defined

Scanned with CamScanner

Code :-

```
→ Class Student{
  public:
    int id;
    string name;

    // Default constructor
    Student(){
        id = 0;
        name = "Soman";
        cout<<"Default constructor called \n";
    }
  void display(){

    cout<<"ID:" <<id<<", Name: "<<name<<endl;
  }
};
    int main(){
→  Student s1; // Default constructor is called
    s1.display();

return 0; }
```

Scanned with CamScanner

Date: / / Page:

② Parameterized Constructor :-

Takes arguments to initialize data.

```
class student{
public:
    int id;
    string name;

student(int i, string n){
    id = i;
    name = n;
cout<<" Parameterized constructor called\n";

void display(){
cout<<" ID "<<id<<", Name: "<<name<<endl;

}
};
int main(){

student s2 (101, "Suman"); // Parameterized constructor
    s2. display();

return 0;
}
```

Scanned with CamScanner

Date: / / Page:

③ Copy Constructor :-

Used to create a copy of an existing object.

```
→ Class Student {
public:
    int id;
    string name;

Student (int i, string n) {
    id = i;
    name = n;
}
 // copy constructor

→ Student (const student &s) {
    id = s.id;
    name = s.name;
    cout << " Copy constructor called \n";
}
void display() {
    cout << " ID: " << id << ", Name: " << name << endl;
}
};
 int main() {
→ Student S1(102, "Suman");
  Student S2 = S1; // Copy constructor is called
  S2.display();

return 0;
}
```

Scanned with CamScanner

Date: / / Page:

⁂ Destructor:-

(i) Destructor is A special function with the same name as class but prefixed with ~

(ii) Called automatically when object goes out of scope

(iii) Used for cleanup like memory release, file close etc.

⁂ key Points:-

(i) No arguments
(ii) No return type
(iii) Only one destructor per class
(iv) Cannot be overloaded.

⁂ Code:-

```
class Student {
public:
    Student(){
      cout << " constructor called \n";
      }
   ~ Student(){
      cout << " Destructor called \n";
      }
  };

int main(){
  Student S1; // constructor called
} // Destructor is automatically called at the
                end of scope.
```

Scanned with CamScanner

Date: / / Page:

(4) Constructor Overloading :-

Constructor overloading means having multiple constructors in the same class with different parameters.

```
class Student {
    string name;
    int age;
public:
    Student(){                          // Default Constructor
    name = "Unknown";
    age = 0;
    }
    Student(string n, int a){           // Parameterized Constructor
    name = n;
    age = a;
    }
    void display(){
    cout << "Name:" << name << ", Age:" << age << endl;
    }
};
int main(){
Student s1;                             // calls default constructor
Student s2("Suman", 23);                // calls parameterized -
                                           constructor
s1.display();
s2.display();

return 0;
}
```

Scanned with CamScanner

✡ this pointer :-

this is keyword in c++. It is a pointer that refers to the current object (the object that is calling the member function).

✡ Why Use this Pointer :-

(i) To refer to current object's members (especially when parameter names are same as variables.)
(ii) To return the current object from a member function.
(iii) To pass current object as argument.

Code :-

```
class Student{
    string name;
public:
→       student (string name){
            this→name=name; // 'this→name' refers
                            to the member variable
        }
void display(){
    cout<<"Name :"<<this→name<<endl;
}
};
int main(){

→   Student s("Suman");
    s.display();

    return 0;
}
```

Scanned with CamScanner

Static Data Member :-

(i) It Belongs to the class, not to any object.
(ii) Shared across all objects (only 1 copy exists).
(iii) Must be declared inside the class & defined outside.

```
#include <iostream>
using namespace std;

class Student {
public:
    static int count;

    Student() { count++; }

    static void showCount() {
        cout << " Total: " << count << endl;
    }
};
int Student :: count = 0;

int main() {

    Student a, b;
    Student :: showCount(); // Total : 2

    return 0;
}
```

Scanned with CamScanner

Date: / / Page:

✱ Friend Function :-

A friend function is a non-member function that has the right to access private & protected members of class.

→ Even though, it's not inside the class, the class given it special permission.

✱ Why Use Friend Function :-

By default, private & protected members of a class cannot be accessed outside the class.

→ But is some cases (like operator overloading, two-class operations, or external helper functions), we may want a non-member function to access private data.

That's when we use friend functions.

✱ Syntax :-

```
class ClassName {

    friend returnType functionName(parameters);

};
```

Scanned with CamScanner

Date: / / Page :

Key Points :-

Feature	Description
(i) Inside class?	✗ Not a class member
(ii) Access to private members?	✓ Yes (if declared friend)
(iii) Uses this pointer?	✗ No
(iv) Inheritence involved?	✗ No

→ Code :-

```
class Box {
private :
    int length;
public :
    Box() : length(0) { }
// Friend function declaration.
→ friend void setLength(Box &b, int len);
};

// Friend function definition (outside the class)
→ void setLength(Box &b, int len) {
    b.length = len;
    cout << "Length set to: " << b.length << endl;
}
int main() {
    Box b1;
→   setLength(b1, 25); // Accessing private member using
                       friend function
    return 0; }
```

When to Use :-

(i) When external functions need access to private data.

(ii) For operator overloading.

Scanned with CamScanner

⛤ Encapsulation :-

Wrapping data (variables) & the code (functions) that operate on the data into a single unit.

In Short :-

Data + functions = class
Access Control = Encapsulation.

⛤ key idea :-

Encapsulation is achieved using:-

(i) A class to wrap data & function.
(ii) Access Specifiers :- private, public, protected

"By making data member private & using public functions to access/modify them, we encapsulate them."

⛤ Syntax & Example :-

```
class Student {
private :
    string name;
    int age;
public:        // public function to set data
    void setDetails(string n, int a){
        name = n;
        age = a;
    };
```

Scanned with CamScanner

// public function to get data
→ void display() {
cout << "Name: " << name << ", Age: " << age << endl;
}
};

int main() {

→ Student S1;
S1.setDetails ("Suman", 22); // Allowed via public method
S1.display();

//S1.name = "Hacked"; → Not allowed (name is private)

return 0;
}

⭐ output :-
Name: Suman, Age: 22

Scanned with CamScanner

Date: / / Page:

Inheritance:-

Acquiring the properties from the parent class.

or :-

Inheritance is a Mechanism where one class inherits properties & behaviours (methods) from parents (another) class.

Syntax:-

```
class Base {
    // base class members
};
class Derived : access_modifier Base {
    // derived class members
};
```

Types of Inheritance in C++ :-

(i) Single Inheritance
(ii) Multiple Inheritance
(iii) Multilevel Inheritance
(iv) Hierarchical Inheritance
(v) Hybrid Inheritance

Scanned with CamScanner

Date: / / Page:

→ Example:-

```
#include<iostream>
using namespace std;

// Base class
→ class Animal{
  public:
      void eat(){
      cout<<"Eating..."<<endl;
      }
  };
// Derived class
→ class Dog: public Animal{
  public:
      void bark(){
      cout<<"Barking..."<<endl;
  }
};

int main(){

→   Dog d;
    d.eat(); // Inherited from Animal
    d.bark(); // Defined in Dog
  return 0;

  }
```

Scanned with CamScanner

① Single Inheritance:-

One class derives from a single base class.
Used where there is a direct "is-a" relationship.

→ Employee
↓
Developer

→
```
class Employee {
public:
    string name; int id;

    void displayInfo(){
    cout<<"Name:"<<name<<", ID:"<<id<<endl;
    }};
```

→
```
class Manager : public Employee{
public:
    void manageTeam(){
    cout<<" Managing team ..."<<endl;
    }};
```

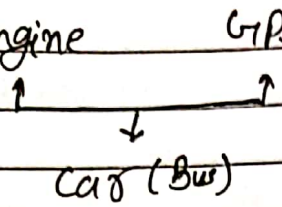
→
```
int main(){
    Manager m;
    m.name = "Suman";
    m.id   = 101;
    m.displayInfo();
    m.manageteam();
}
```

Scanned with CamScanner

Date: / / Page:

② Multiple Inheritance:-

A class inherits from two or more base classes.

→ 

→
```
class Engine{
public:
    void startEngine(){
    cout<<" Engine Started."<<endl;
}};
```

→
```
class GPS{
public:
        void showLocation(){
    cout<<"Showing GPS location."<<endl;
}};
```

→
```
class Car: public Engine, public GPS{
public:
        void drive(){
            cout<<" Car is driving."<<endl;
    }};
```

→
```
int main(){
    Car c;
    c.startEngine();
    c.showLocation();
    c.drive();
}
```

Scanned with CamScanner

Date: / / Page:

③ Multilevel Inheritance:—

A class inherits from another derived class, forming a chain of inheritance.

→ Person → Student → Graduate.

→
```
class Person{
public:
    string name; int roll;

    void showName(){
      cout<<"Name :"<<name<<endl;
}};
```

→
```
class Student : public Person{
public:
    void showRoll(){
    cout<<" Roll No." <<roll<<endl;
}};
```

→
```
class Graduate: public Student{
public:
    void showDegree(){
      cout<<" Graduation Completed."<<endl;
}};
```

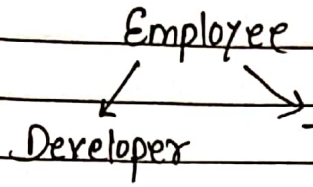
→
```
int main(){
Graduate g;
    g.name = "Suman";
    g.roll = 22;
      g.showName(); g.showRoll(); g.showDegree();
}
```

Scanned with CamScanner

Date: / / Page :

④ Hierarchical Inheritance :-

→ Multiple derived classes inherit from the same base class.



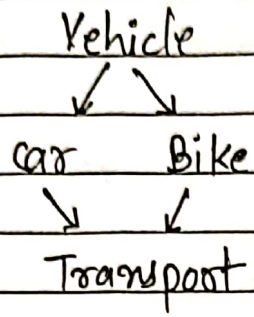

```
→ class Employee{
public:
   void work(){
   cout<<" employee working..." <<endl;
};

→ class Developer : public Employee{
public:
    void code(){
    cout<<" Developer writing code. "<<endl;
};

→ class Tester : public Employee{
public:
    void test(){
   cout<<" Tester testing application." <<endl;
};

→ int main(){
Developer d; Tester t;
d.work(); d.code();
t.work(); t.test();
}
```

Scanned with CamScanner

Date: / / Page:

⑤ Hybrid Inheritance :-

→ Combination of more than one type of inheritance (e.g. Multiple + Hierarchical).



→
```
class Vehicle {
public:
    void start(){
    cout<<" Vehicle starting ..."<<endl;
};
```

→
```
class Car: public Vehicle {
public:
    void drive(){
    cout<<" Driving car. "<<endl;
};
```

→
```
class Bike : public Vehicle{
public:
    void ride(){
     cout<<" Riding bike."<<endl;
  };
```

→
```
class Transport : public Car, public Bike {
public:
    void travel(){
    cout<<" Using transport. "<<endl; };
```

Scanned with CamScanner

→ int main(){

Transpoot t;

t.start(); // Ambiguity: which Vehicle copy to use?
t.drive();
t.ride();
t.travel();
}

Note :- Without virtual Inheritance, "Transpoot" would get two copies of "Vehicle's" members.

→ Problem :- "Vehicle" start function is called twice because two copies exist - one from "Car" & one from "Bike".

→ Solution :- With Virtual Inheritance :-

→ class Vehicle {
public:
void start(){
cout<<" Vehicle starting---"<<endl;
};

→ class Car : virtual public Vehicle{
public:
void drive(){
cout<<" Driving car."<<endl;
};

Scanned with CamScanner

→ class Bike : virtual public Vehicle{
public:
    void ride(){
    cout<<"Riding bike."<<endl;
}
};

→ class Transport : public Car, public Bike{
public:
    void travel(){
    cout<<" Using transport."<<endl;
}
};

→ int main(){

    Transport t;
    t.start(); // No ambiguity due to virtual inheritance.
    t.drive();
    t.ride();
    t.travel();

}

→ Use virtual public Vehicle in both Car & Bike to ensure only one shared copy of Vehicle exists.

Scanned with CamScanner

⁂ Abstraction :-

Abstraction is the process of hiding implementation details & only showing essential features to the user.

→ Example in real life :-

① You drive a car using the steering wheel & pedals.

You don't need to know how the engine, clutch, or fuel injection work.

⁂ Why Abstraction :-

① To reduce Complexity.
② To improve security (hide sensitive parts)
③ To write flexible, scalable code.
④ To define a general interface for related classes.

⁂ How to Achieve Abstraction :-

① Encapsulation (Basic level) :-

Using access modifiers (private, public) to hide data.

② Abstract classes + Pure Virtual Functions (Advanced level) :-

ⓐ Defining a function with virtual void functionName() = 0; make it pure virtual.

ⓑ A class with at least one pure virtual function is called an abstract class.

Scanned with CamScanner

ⓒ You cannot create objects of an abstract class.

ⓓ Derived classes must implement all pure virtual functions.

⊛ Virtual function :-

A virtual function is a function in the base class that you expect to override in derived classes.

→
```
class Base{
public:
    virtual void show(){
    cout<<" Base show"<<endl;
    }
};
```

Note:- It allows runtime polymorphism when using pointers/references.

⊛ Pure Virtual Function :-

A pure virtual function is a function that has no body in the base class & must be implemented in derived class.

→
```
class Base{
public:
    virtual void show()=0; // Pure virtual
};
```

Scanned with CamScanner

Date: / / Page :

Example :-

```
class Shape {
public:
    // Pure virtual function
    virtual void draw() = 0;

void info() { cout << "This is a shape";

};

class Circle : public Shape {
public:
    void draw() override {
     cout << "Drawing a circle";
    }
};

class Square : public Shape {
public:
    void draw() override {
     cout << "Drawing a Square";
    }
};

int main() {
// Direct object creation of derived classes
 Circle c;
 Square s;

 c.draw(); // output :- Drawing a Circle
 s.draw(); // output :- Drawing a Square.

}
```

Scanned with CamScanner

Date: / / Page:

⭐ Polymorphism :-

One entity play multiple role based on the situation.

→ The word polymorphism comes from Greek :-

Poly → many
Morph → forms

⭐ Types of Polymorphism :-

① Compile - time Polymorphism
② Run - time Polymorphism

① Compile - time Polymorphism :-

Also called static polymorphism & Early Binding. Behavior is decided at compile time.

→ Example :-

ⓐ Function Overloading :-

Same function name, different parameter list.

ⓑ Operator Overloading :-

Same operator, different meaning for user - defined types.

Scanned with CamScanner

② Runtime Polymorphism :-

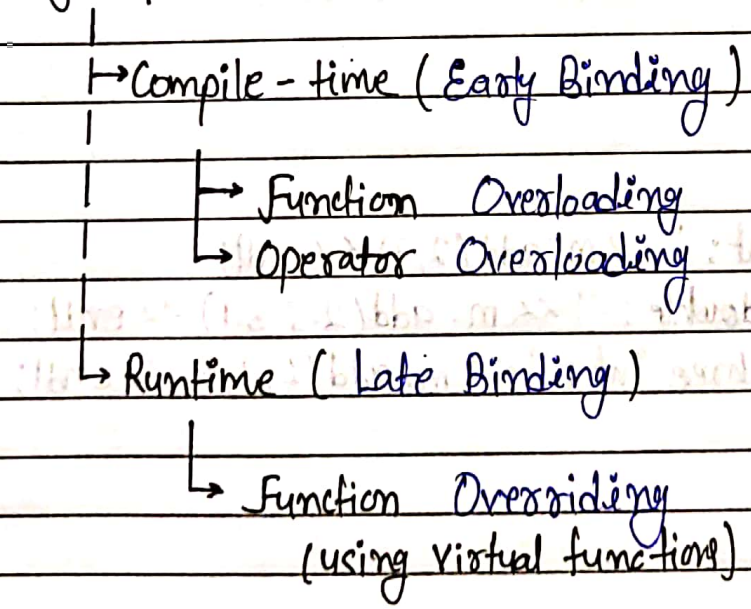
Also called Late Binding / Dynamic Polymorphism. Behavior is decided at runtime using virtual functions & base class pointers/references.

→ Example :-

Function Overriding :-

Derived class provides its own implementation of a base class's virtual function.

④ Diagram Representation :-



Scanned with CamScanner

Date: / / Page:

Example :-

→ **Compile-time Polymorphism (Function Overloading):-**

```
#include <iostream>
using namespace std;

class Math {
public:
    int add(int a, int b) return a+b;

    double add(double a, double b) return a+b;

    int add(int a, int b, int c) return a+b+c;

};

int main() {
    Math m;

    cout<<"Add int: "<<m.add(2,3)<<endl;
    cout<<"Add double: "<<m.add(2.5,3.1)<<endl;
    cout<<"Add three int: "<<m.add(1,2,3)<<endl;

    return 0;
}
```

Scanned with CamScanner

Date: / / Page:

⭐ Compile-time Polymorphism (Operator Overloading):-

```
#include <iostream>
using namespace std;

class Complex{          // 4+6i → (iota)
public:
    float real, imag;

    Complex(float r, float i){
    real = r; imag = i;}

    // Overload + operator
    Complex operator+(const Complex &c){
      return Complex(real + c.real, imag + c.imag);
    }
};

int main(){
  Complex c1(2.5, 3.5), c2(1.5, 2.5);

  Complex c3 = c1 + c2; // calls overloaded +

  cout << "Result:" << c3.real << " + " << c3.imag << "i" << endl;

}
```

Real part

Imaginar part

Scanned with CamScanner

Date / / Page:

⊛ Runtime Polymorphism (Function Overriding):-

```
#include <iostream>
using namespace std;

→ class Payment {
public:
    virtual void process() {
    cout << " Processing payment" << endl;
    }
};

→ class CreditCard : public Payment {
 public:
     void process() override {
    cout << " Processing Credit Card Payment" << endl;
}};

→ class Paypal : public Payment {
 public:
     void process() override {
     cout << " Processing PayPal Payment" << endl;
}};

→ int main() {
 Payment * P;
 CreditCard cc;
 PayPal PP;

 P = &cc;   P → process();
 P = &PP;   P → process(); }
```

Scanned with CamScanner

Practice Question :-

(i) Write a program to create a class with private data members & a friend function that can access those members to perform an operation.

(ii) Design a class to present a bank account. Initialize account details using a constructor & display values.

(iii) Create an abstract base class for shapes with a pure virtual function to calculate area. Derive multiple shape classes (like circle, rectangle).

(iv) Implement an employee management system with a base class Employee & derived classes for different roles. (e.g., Manager, Developer). Each derived class should have extra data members & display all details.

(v) Write a program with a base class "Bank" that has a virtual function. Override this function in multiple derived classes to produce different outputs, & demonstrate calling them using base class pointers or references.

Scanned with CamScanner

Date: / / Page:

⭐ Exception Handling:-

Exception handling in C++ is a mechanism to detect & handle runtime errors in a controlled way, so that the program doesn't crash unexpectedly.

→ It uses three main keywords:-

① **try:-** Block of code where you place risky operations (that might throw an error)

② **throw:-** Used to signal (or "throw") an exception when an error occurs.

③ **catch:-** Block of code that handles the thrown exception.

⭐ Syntax:-

```
int main(){
try{
    int a=10, b=0;
    if(b==0)
        throw "Division by zero error!" //throw an exception
    cout<<a/b<<endl;
}
    catch (const char* msg) {
        cout << "Error: "<<msg<<endl;
    }
    return 0;
}
```

Scanned with CamScanner

→ How It Works :-

(I) Code inside the "try" block runs normally until an error is found.
(II) If "throw" is called, control jumps immediately to the matching "catch" block.
(III) If no matching "catch" is found, the program terminates with an error.

(IV) Catching Different Types of Exceptions :-

```
#include<iostream>
using namespace std;
int main() {
try{
        int code=2;
    if(code==1) throw 404;    // int exception
  else if(code==2) throw 3.14;    // double exception
    else throw "Unknown error";  // string exception
}
catch(int e){
   cout<<" Integer error code:"<<e<<endl;
}
catch(double e){
   cout<<" Double error value:"<<e<<endl;
}
catch(const char*e){
   cout<<" String error: "<<e<<endl;
}
}
```

Scanned with CamScanner

Date: / / Page:

(A) STL (Standard Template Library):-

The standard Template Library (STL) in C++ is a collection of pre-written generic classes & functions that save time & effort in programming.

It mainly includes containers, algorithms, iterators & functors.

→ Note:- Think of STL as a ready-made toolkit for data structures & algorithms.

(A) Containers:- Containers are templates that hold data in specific way.

Ex:- (I) vector → dynamic array.
(II) set → sorted unique elements.
(III) map → sorted key-value pairs, unique keys.
(IV) unordered_map → unsorted key-value pairs.

(I) Vector:-

A vector is a dynamic array in C++ that can automatically grow ~~then~~ when more elements are inserted than its current capacity.

→ When the vector's size exceeds its capacity, it allocates new memory (usually double the old capacity) & copies existing element there.

→ Header File:-

```
#include <vector>
```

Scanned with CamScanner

Date: / / Page:

⭐ Key Features :-

(i) Contiguous memory allocation (like arrays, supports random access in O(1) time).
(ii) Dynamic resizing (capacity usually doubles when needed.)
(iii) Efficient insertions at the end: O(1) time.
(iv) Slower insertions in middle/start: O(n) due to shifting elements

⭐ Syntax :-

```
vector<int> v;          // Empty vector
vector<int> v(5);       // size 5, default values 0
vector<int> v(5,10);    // size 5, all values 10
vector<int> v2 = {1,2,3}; // Initializer list.
```

⭐ Common functions :-

	functions	Description
(i)	push_back(x)	Add element at end
(ii)	pop_back()	Remove last element
(iii)	size()	Number of elements
(iv)	capacity()	Allocated storage size (may be >= size)
(v)	clear()	Remove all elements
(vi)	empty()	Check if vector is empty
(vii)	erase(pos)	Remove at position.

Scanned with CamScanner

ⓐ Iterating Through a Vector :-

→ vector<int> v = {5, 10, 15};

// Range-based loop

→ for (int element : v) cout << x << " ";

// Iterator loop

```
for(auto it : v) {
        cout << it << endl;
    }
```

ⓐ Sorting a Vector :-

vector<int> v = {1, 4, 8, 5};

sort(v.begin(), v.end()); // Ascending order

reverse(v.begin(), v.end()); // Descending order.

Scanned with CamScanner

Page / / Date: / / Page.

② Set :-

A set is an associative container in C++ that stores only unique elements in sorted order (by default ascending) using a balanced binary search tree (usually Red-Black Tree).

→ **Header file:-**

```
#include<set>
```

⭐ Key features :-

(i) Unique elements only
(ii) Sorted order
(iii) Automatic Ordering → no need to manually sort
(iv) Fast search, insertion, & deletion: O(log n) time complexity.

⭐ Syntax :-

```
set<int>S;                  // Empty set
set<int>S = {10,20,30};     // Initializer list
set<string> name;           // String set
```

⭐ Common Functions :-

	Function	Description
(i)	insert(x)	Adds element (ignored if already present)
(ii)	erase(x)	Remove element by value
(iii)	find(x)	Returns iterator to element or end() if not found
(iv)	count(x)	Returns 1 if element exists, else 0
(v)	size()	Returns number of elements
(vi)	empty()	checks if set is empty
(vii)	clear()	Removes all elements.

Scanned with CamScanner

Date: / / Page:

Example:-

```
# include <iostream>
# include <set>
using namespace std;

int main(){
  set<int>S;
// Insert elements

  S.insert(5);
  S.insert(1);
  S.insert(3);
  S.insert(5); // Duplicate ignored.

// Display elements (sorted)

  for(int x:S) cout<<x<<" ";

// Search
  if(S.find(3)!S.end()) cout<<"\n Found 3";

// Erase
  S.erase(1);

  cout<<"\n After erasing 1: ";

  for(int x:S) cout<<x<<" ";

}
```

Scanned with CamScanner

③ map :-

A map is an associative container in C++ that stores key-value pairs in sorted order of keys (by default ascending).

ⓐ Keys are unique (no duplicates allowed).
ⓑ Values can be duplicate

⁂ Header file :-

```
#include<map>
```

⁂ Key features :-

① Stores (key, value) pairs.
② keys are always sorted (ascending by default)
③ Fast insertion, deletion, & search in O(logn) time.
④ Duplicate keys are not allowed.

⁂ Syntax :-

```
map<keyType, valueType>m;

map<string, int> marks;

map<int, string, greater<int>> m2; // Descending order by key
```

Scanned with CamScanner

Date: / / Page

Common Functions :-

	function	Description
(I)	insert({key, value})	Insert a new key-value pair
(II)	m[key] = value	Insert or update value for key.
(III)	erase(key)	Remove element by key.
(IV)	erase(it)	Remove element by iterator.
(V)	find(key)	Returns iterator to element or end()
(VI)	size()	Number of elements
(VII)	empty()	checks if map is empty
(VIII)	clear()	Removes all elements.

→ Example :-

```
#include<bits/stdc++.h>
using namespace std;

int main(){
  map<string, int> marks;

  // Insertion methods
  marks["Alice"] = 90;          // using []
  marks.insert({"Bob", 85});    // using insert()
  marks["Charlie"] = 95;

  // Traversing map

  for(auto &p : marks){
  cout<<p.first<<" => "<<p.second<<endl;
  }
```

Scanned with CamScanner

// Search

```
if( marks.find("Bob") != marks.end())
    cout<<"Bob found with marks"<<marks["Bob"]<<endl;
```

// Erase

```
marks.erase("Alice");
}
```

④ unordered_map :-

An unordered_map is an associative container that stores key-values pairs but does not maintain any order of keys.

ⓐ It is implemented using a hash table
ⓑ keys are unique (no duplicate)
ⓒ Provides average O(1) insertion, deletion, & search.

④ Header file :-

```
#include<unordered_map>
```

④ Syntax :-

```
unordered_map<keyType, Value> m;
unordered_map<string, int> marks;
```

Note :- Functions are same as ordered [Normal] map no difference at all.

Scanned with CamScanner

→ https://www.youtube.com/@webdermastery18.

Date: / / Page:

⊗ Example :-

```
#include<bits/stdc++.h>
using namespace std;

int main(){
    vector<int> nums = {1,2,2,3,1,4,2,3,3};

    unordered_map<int, int> freq;

    // Count frequency

    for(int num: nums){
        freq[num]++; // Increase count for this number
    }

// Print frequencies

    cout<<"Element → frequency \n";
    for(auto &p: freq){
    cout<<p.first<<"→"<<p.second<<endl;
}
return 0;
}
```

"Thank you So Much for Watching! Please like, share & Subscribe" ☺

"Happy Coding, Happy Hacking"

Scanned with CamScanner